hashlock.

Security Audit

WORM (DeFi)

November 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

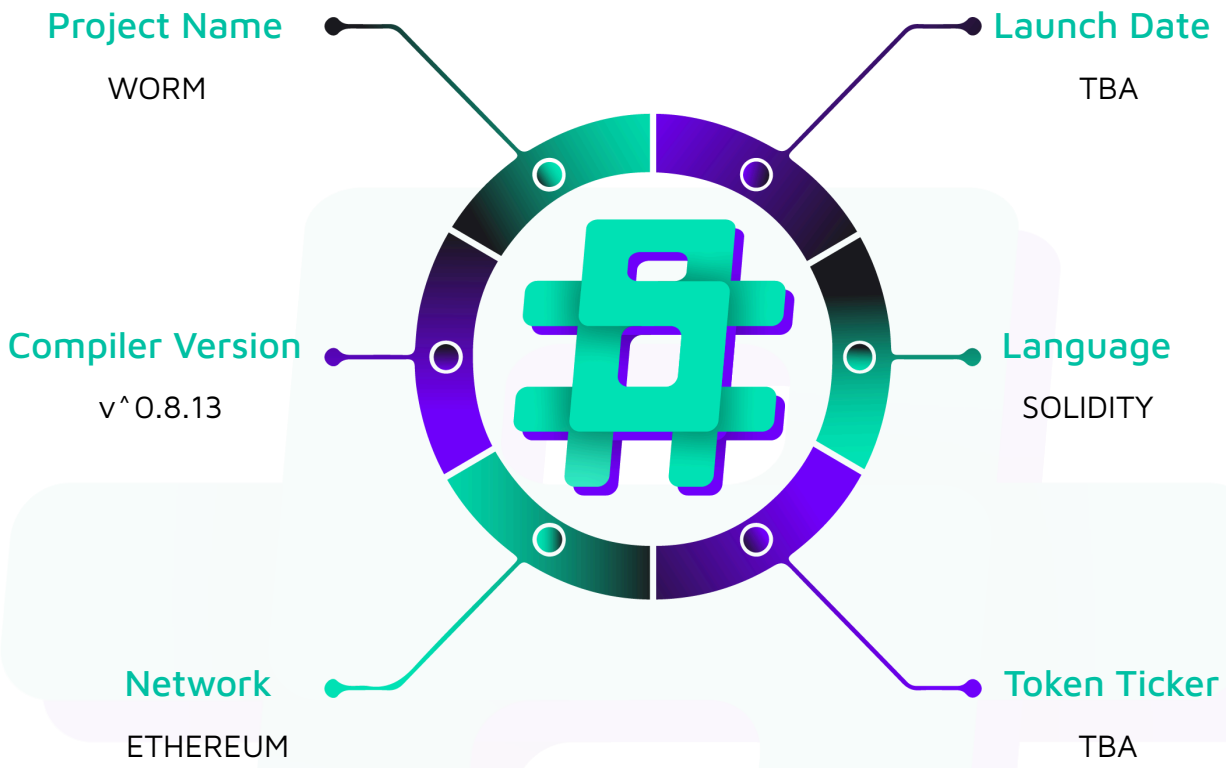
The WORM team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

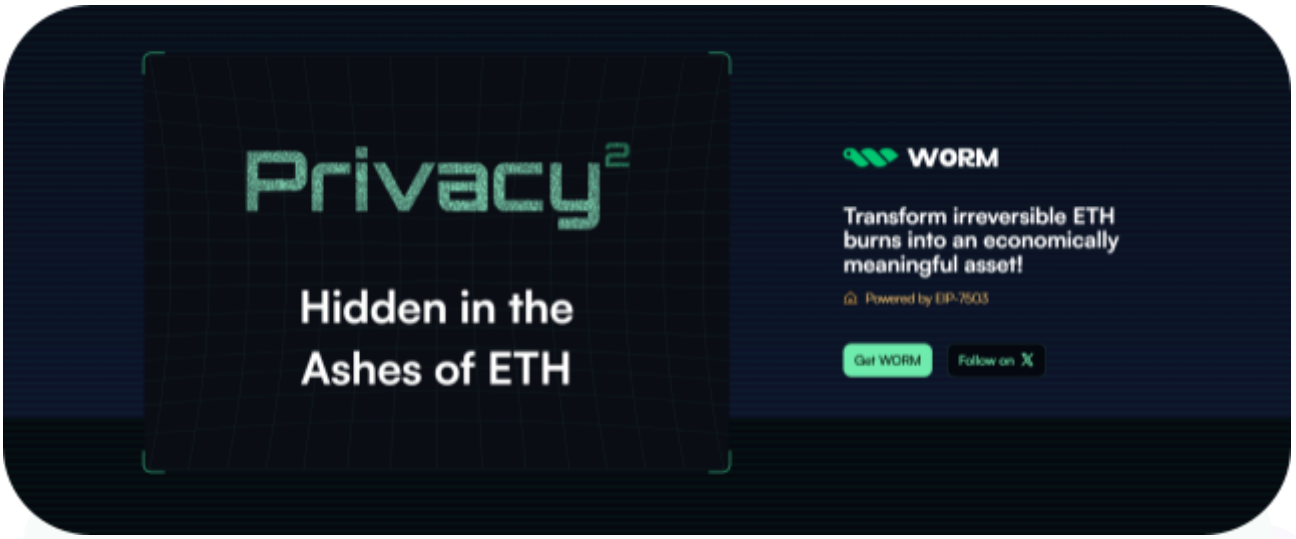
WORM is an ERC-20 privacy-focused token on Ethereum that introduces a new crypto-economic model based on *private proof-of-burn*. Users irreversibly burn ETH through a zero-knowledge (zk-SNARK) mechanism, and in return receive WORM, a scarce token whose issuance is provably backed by destroyed ETH. This creates a new primitive for decentralized finance and privacy, enabling "real value" minting without modifying Ethereum's base layer. The project positions itself as a decentralized privacy + cryptographic scarcity protocol, rather than a typical utility token or stablecoin.

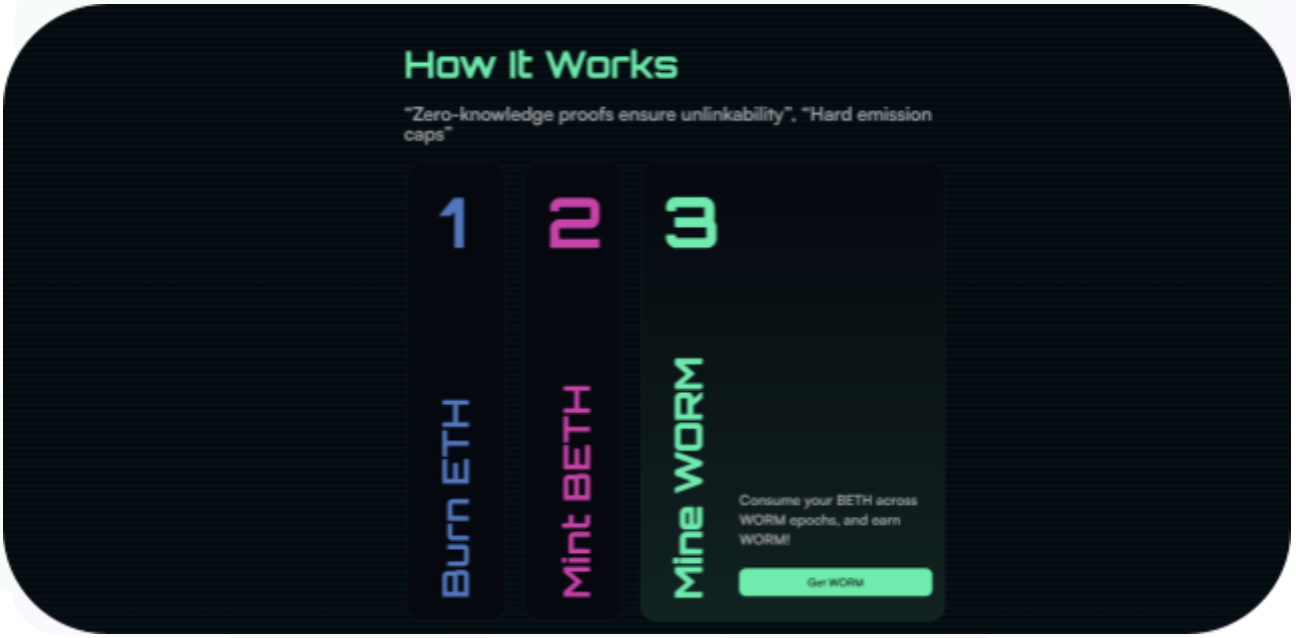
Project Name: WORM
Project Type: Defi
Compiler Version: ^0.8.13
Website: https://worm.cx/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the WORM project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**WORM Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**November, 2025**
Contract 1	BETH.sol
Contract 2	WORM.sol
Contract 3	Staking.sol
Audited GitHub Commit Hash	0134fb6ca0680fbff32e32b152b669c6ab5af066
Fix Review GitHub Commit Hash	ca123363a7e43e68c7022cec1291ea7827161494

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 High severity vulnerabilities

4 Low severity vulnerabilities

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
BETH.sol - Allows users to burn ETH and reveal BETH through the verification of ZK proofs - Users, broadcasters, and provers may execute hooks during the minting process to handle their received BETH - 0.5% of all BETH minting and revealing is forwarded to the staking pool for WORM staking rewards	**Contract achieves this functionality.**
WORM.sol - Allows BETH holders to transmute their tokens into WORM - WORM rewards are proportionally distributed every epoch lasting 10 minutes	**Contract achieves this functionality.**
Staking.sol - Allows WORM holders to stake WORM for proportional BETH rewards based on weekly epochs	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the WORM project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the WORM project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] BETH#spendCoin - Complete loss of BETH when spendCoin() is called more than once

Description

The owner set during `mintCoin()` never changes. However, `_spendParams.coin` changes on each `spendCoin()` call. Therefore, any call of `spendCoin()` beyond the initial call causes the `coinOwner` mapping to access a non-existent owner, as it's not referencing the original root coin, ultimately sending all revealed BETH tokens to `address(0)`.

Vulnerability Details

There is a concept of encrypted coins acting as private metadata to track the BETH minting and revealing process. The `spend circuit` specifies these coins as `Poseidon3(POSEIDON_COIN_PREFIX, burnKey, remainingBalance)`. Throughout the trace, we will be referencing coins in the simplified format of:

`HASH(constant, burnKey, remainingBalance)`.

1. Call `mintCoin()`

Alice wants to mint a total of `5 BETH` for the genesis `mintCoin()` call. She decides to reveal `3` of the `5 ETH`, leaving her with a remaining coin balance of `2 ETH`.

- `_mintParams.remainingCoin = HASH(constant, burnkey, 2 ETH)`
- `coinSource[HASH(constant, burnkey, 2 ETH)] = HASH(constant, burnkey, 2 ETH)`
- `coinRevealed[HASH(constant, burnkey, 2 ETH)] = 3 (3 ETH has been revealed so far)`
- `coinOwner[HASH(constant, burnkey, 2 ETH)] = ALICE`

2. Call `spendCoin()`

Alice wants to reveal another `1.5 ETH` via `spendCoin()`, revealing a total of `4.5 ETH`, or 1.5 out of the 2 ETH left from the remaining coin.

- `_spendParams.coin = HASH(constant, burnkey, 2 ETH)`
- `rootCoin = coinSource[HASH(constant, burnkey, 2 ETH)] = HASH(constant, burnkey, 2 ETH)`
- `owner = coinOwner[HASH(constant, burnkey, 2 ETH)] = ALICE`
- `rootCoin != 0 is true`
- `coinSource[_spendParams.remainingCoin] == 0 is true. This is because _spendParams.remainingCoin` = HASH(constant, burnkey, 0.5 ETH), because Alice wants to reveal 1.5 out of the 2 ETH left.`
- `coinSource[HASH(constant, burnkey, 2 ETH)] = 0`
- `coinSource[HASH(constant, burnkey, 0.5 ETH)] = HASH(constant, burnkey, 2 ETH)`
- `coinRevealed[HASH(constant, burnkey, 2 ETH)] = 3 ETH + 1.5 ETH now revealed, = 4.5 ETH.`

3. Call `spendCoin()` Again

Alice wants to reveal another `0.2 ETH`. If the original `2 ETH` coin is used, the tx reverts as expected:

- `_spendParams.coin = HASH(constant, burnkey, 2 ETH)`
- `rootCoin = coinSource[HASH(constant, burnkey, 2 ETH)] = 0`
- `rootCoin != 0 immediately fails, as it was previously destroyed.`
- `This means _spendParams.coin must be pointing to the current newer coin which is HASH(constant, burnkey, 0.5 ETH).`

This confirms the correct coin needs to be `HASH(constant, burnkey, 0.5 ETH)`:

- `_spendParams.coin = HASH(constant, burnkey, 0.5 ETH)`
- `rootCoin = coinSource[HASH(constant, burnkey, 0.5 ETH)] = HASH(constant, burnkey, 2 ETH), correctly grabbing the root coin.`
- `owner = coinOwner[HASH(constant, burnkey, 0.5 ETH)] = address(0)`
- `rootCoin != 0 is true`

- `coinSource[_spendParams.remainingCoin] == 0 is true, because _spendParams.remainingCoin` = `HASH(constant, burnkey, 0.3 ETH) is fresh, because Alice wants to reveal 0.2 out of the remaining 0.5 ETH left.`
- `coinSource[HASH(constant, burnkey, 0.5 ETH)] = 0`
- `coinSource[HASH(constant, burnkey, 0.3 ETH)] = HASH(constant, burnkey, 2 ETH), correctly attaching newer coin to the root coin.`
- `coinRevealed[HASH(constant, burnkey, 2 ETH)]` = 4.5 ETH + 0.2 ETH now revealed, = 4.7 ETH`
- `_mint(address(0), 0.2 ETH - fees)`
- `Alice's BETH to be revealed gets minted to address(0) instead of Alice.`

Impact

Complete loss of revealed BETH for users calling `spendCoin()` more than once.

Recommendation

During `spendCoin()`, access the owner via `coinOwner[rootCoin]` instead of `coinOwner[_spendParams.coin]`.

Status

Resolved

[H-02] proof_of_burn.circom - Proof Generation Failure From Non-Exact Burn Address ETH Balance Leads to BETH Minting DoS

Description

When a user attempts to generate a proof when the amount of `ETH` in their burn address does not exactly match the revealed amount set during address generation, the proof generation fails.

Note: This vulnerability involves the proof of burn circuit logic and frontend proof generation, outside of the immediate domain of this specific audit, but has been included due to its severity.

Vulnerability Details

1. Alice clicks `Generate Address`
2. Alice clicks `Burn ETH`, specifying an amount of `0.001 ETH` to burn. Alice clicks burn, sending the `ETH` amount to the burn address.
3. A malicious `MEV` bot filters the mempool for all fresh addresses with a balance of 0 and nonce of 0, looking for a single `ETH` transfer to fresh addresses every block. They backrun the burn `tx`, sending 1 wei to the burn address. The burn address now has more than `0.001 ETH`. Their attack is successful as long as the dust `ETH` is sent before the proof can be generated.
4. Alice clicks `Mint` and `Generate proof`. The job attempts to execute and the proof generation fails with the error string "Proof generation failed. Job failed: No `ETH` present in the burn address."

The underlying `proof of burn` template does solve dust attacks that donate dust `ETH` amounts to the user's burn address, by checking the pure `intendedBalance` of exactly `0.001 ETH` without any dust is less than or equal to the `actualBalance`, which can be arbitrarily larger than the intended balance due to malicious donations. Therefore, the vulnerability likely originates from how the backend handles the data needed to generate proofs. The frontend likely has one or more of the following problems:

- Explicitly checking the current `ETH` balance in the burn address is exactly the original revealed amount set on the frontend, or otherwise any kind of pre-balance check that doesn't even allow for the start of the proof generation
- Uses the current dusted `ETH` balance of `0.001+1` wei `ETH` as the `revealAmount`, while the `intendedBalance` is static at `0.001 ETH`, causing `constraint failure`, anytime the dusted amount exceeds the intended balance.
- A potentially even more fundamental problem, where verifying the `MPT` leaf somehow requires an exact initial amount in the balance, and once it changes, it's permanently bricked. Unlikely, but should be noted.

Impact

A single `MEV` bot can trivially cause all user proof generation to fail by dusting `1` wei to all burn addresses that have been funded but not yet minted `BETH`.

Recommendation

Ensure the frontend gathers all relative variables such as `revealAmount`, `intendedBalance`, and `actualBalance` properly. Additionally, allowing the user to set a lower intended balance may be a necessary step. It's important to note that all donation attempts must be stopped. Meaning, the frontend needs to solidify all `ETH` amount values related to the proof of burn. E.g. `intendedBalance` , `revealAmount`, and `actualBalance` need to be cached in such a way that is agnostic to any dust donations, anytime the `generate proof` button is called.

Lastly, even though the proof is generated off chain, a particularly malicious bot could donate 1 wei every block, and if the frontend doesn't lock in valid values or handle this correctly, it may continue to indefinitely `DoS` the proof generation, depending on how values are used.

Note

The circuit already allow to prove an intended-balance even when the target account contains dust. The issue is on the frontend side. Will be fixed.

Status

Resolved

Low

[L-01] BETH#mintCoin - Frontrunnable Broadcasts Leading to Loss of Gas and Broadcasting Fee

Description

Honest broadcasters can be frontrun by `MEV` bots, losing the gas spent on the failed transaction and their broadcasting fee.

Vulnerability Details

Anyone with access to valid calldata can call `BETH.mintCoin()` on behalf of a burner looking to mint `BETH`. This includes malicious `MEV` bots viewing the mempool for `mintCoin()` transactions. Because the broadcaster is `msg.sender`, and not explicitly baked into the proof commitment, as long as the gas cost is less than the `BETH` earned by the `broadcasterFee`, `MEV` bots can be set up to profit by frontrunning honest broadcasters.

Notably, the higher the `broadcasterFee`, the more incentive a `MEV` bot will have to frontrun that particular broadcaster.

Impact

Honest broadcasters that are frontrun by `MEV` lose the gas spent on the transaction along with the potential broadcasting fee that would have been earned.

Recommendation

Broadcasters cannot be baked into the proof commitment, as this then puts too much reliance on a single untrusted actor to carry out a future duty. Warn broadcasters that their transactions may be frontrun, and to use private RPCs like `flashbots protect`, `Llama nodes`, or `CoWSwap's MEV blocker RPC`.

Status

Resolved

[L-02] Staking#lock - Unlimited Max Stake Duration Leads to Loss of Funds

Description

Users can stake for an unlimited number of epochs, causing their `WORM` to be permanently lost.

Vulnerability Details

There is no check on the maximum number of epochs a stake can last. Users can stake up to the number of epochs it takes to reach the Ethereum block gas limit in a single transaction.

Impact

Users can accidentally or intentionally stake indefinitely into the future, essentially burning their `WORM`.

Recommendation

Enforce a maximum stake duration of `1-15` years during `lock()` calls.

```
E.g. require(_numEpochs < 52*4, "stake exceeds 4 years");
```

Status

Resolved

[L-03] WORM#participate - 1 Wei of BETH Participation Yields Entire WORM Epoch Rewards

Description

Users can participate in the transmuting of `BETH` to `WORM` with dust amounts, allowing for disproportionate rewards.

Vulnerability Details

Users can call `WORM.participate()` to dust `1 Wei` of `BETH` into thousands of future epochs, ensuring that if a single epoch has no other participation, they obtain the full `WORM` rewards for that epoch. The calculated `mintAmount` becomes the entire reward for the epoch. If this happens even once, or anytime there is very low participation in a single epoch, this can severely reduce the confidence in the price of `WORM`, due to nearly free or at least heavily discounted `WORM` being created.

Impact

1 wei BETH participation is rewarded with substantial `WORM` tokens.

Recommendation

Introduce a minimum threshold of `BETH` participation for `WORM` to be minted, to guarantee a minimum floor conversion ratio of `WORM:BETH`. It may be best from an economic perspective to make this very aggressive (`E.g. 0.01-1 BETH`).`E.g. require (_amountPerEpoch >= 0.001e18, "too small");` Currently, anytime there is non-zero total participation, `WORM` is minted for that epoch. An even more aggressive approach to consider to guarantee a minimum scarcity of `WORM`, is to disallow minting of `WORM` for an epoch if the `total` is under a threshold. A consequence of this is that multiple participants would be a prerequisite as long as the `total` threshold is larger than the single minimum amount for a user. This adds some complexity and would require refunding users in a mapping via a separate pull operation if the total threshold for an epoch is not met.

Status

Acknowledged

[L-04] Staking#lock - Staking 1 Wei of WORM yields entire BETH Epoch Rewards

Description

Users can lock `1 Wei` of `WORM` in the Staking contract, allowing for disproportionate rewards if no other stakers exist for the epoch, leading to near infinite APR.

Vulnerability Details

`BETH` is rewarded to stakers who lock their `WORM` in `Staking.lock()`. If only a single staker locks `1` Wei of `WORM` for the epoch, they earn the entire `BETH` rewards for that epoch.

If low staking participation occurs for an epoch, this can severely reduce the confidence in the price of `BETH`.

Impact

1 Wei of staked `WORM` is rewarded with substantial `BETH` tokens when it is the sole locked position.

Recommendation

Consider introducing a minimum threshold of `WORM` when staking to guarantee a minimum floor conversion ratio of `WORM:BETH`. It may be best from an economic perspective to make this unusually aggressive.

Status

Acknowledged

QA

[Q-01] BETH#mintCoin - Burner Loses Minted BETH When `revealedAmount` Equals the Middleman Fees

Description

Users can request an amount that matches `prover + pool + broadcaster` fees, paying the middlemen for their work but yielding zero `BETH` for themselves.

Vulnerability Details

When calling `mintCoin()`, a user can request a revealed amount equal to the sum of the middleman fees, passing the minimum amount check of: `_mintParams.proverFee + _mintParams.broadcasterFee <= **revealedAmountAfterFee`.

If the revealed amount matches the total of the fees, then the BETH amount sent to the burner is `0 revealedAmountAfterFee - _mintParams.broadcasterFee - _mintParams.proverFee`.

Impact

Users can footgun themselves into losing small BETH reward amounts.

Recommendation

Warn users that small amounts can be rounded to `0` due to fees, or enforce a minimum revealed amount during `mintCoin()`.

Status

Acknowledged

Centralisation

The WORM project values decentralisation alongside security and utility.

The protocol's functions are designed to operate independently, minimising reliance on the internal team and hardcoded values, while maintaining robust security and functionality.

Conclusion

After Hashlock's analysis, the WORM project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au